INFOMOV-Assignment3 Not a Drill

Chris Lokhorst - 7330081, Roel de Jeu - 2577763

June 2024

1 Introduction

The program we have decided to optimize is a Minecraft clone written in C [1]. Since the program already uses multi-threading for its chunk generation, we are more interested in optimizing the rendering scheme. Focusing on the rendering also allows us to more easily measure performance improvements, as we can simply wait for the chunk generation to be fully finished. The original settings of the program limits the chunk render radius to 16 and limits the fps by VSync. We have increased the render radius to 64 chunks in order to make the program sweat and disabled VSync to allow us to measure a difference in performance. The chunk size can also be modified, but as we felt optimizing this variable is not that interesting, we have decided to keep this at 32.

1.1 Base level Performance

We used the Very Sleepy Profiler in order to gain insight of the bottlenecks application. After waiting for all the chunks in the render distance to render, we landed at about 6 frames per second.

When we looked at the profiler, we noticed one particular function was taking up about 95% of the total processing time. This can be seen in figure 1, the function find_chunk was taking up a lot of the processing power.

Name	Excl...	Incl...	% Exclusi...	% Inclusi...	Module	Source File	So...	Address
mainCRTStartup	0.00s	46.20s	0.00%	100.00%	craft	[unknown]	0	0x7ff7ced8113d
__tmainCRTStartup	0.00s	46.20s	0.00%	100.00%	craft	[unknown]	0	0x7ff7ced8142a
BaseThreadInitThunk	0.00s	46.20s	0.00%	100.00%	KERNEL32	[unknown]	0	0x7ffe8bdb257d
main	0.00s	46.20s	0.00%	100.00%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	2825	0x7ff7ced91ea0
render_chunks	0.02s	44.25s	0.04%	95.79%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1613	0x7ff7ced8c8d8
ensure_chunks	0.00s	43.61s	0.00%	94.40%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1427	0x7ff7ced8be76
ensure_chunks_worker	0.14s	43.61s	0.31%	94.40%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1338	0x7ff7ced8b83c
find_chunk	43.46s	43.46s	94.07%	94.07%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	528	0x7ff7ced87b8f
glfwSwapBuffers	0.00s	1.76s	0.00%	3.82%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	548	0x7ff7cee609f2
_glfwPlatformSwapBuffers	0.00s	1.76s	0.00%	3.82%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	606	0x7ff7cee6a915
DwmFlush	1.76s	1.76s	3.80%	3.80%	dwmapi	[unknown]	0	0x7ffe86684a71
chunk_visible	0.33s	0.33s	0.72%	0.72%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	545	0x7ff7ced87c6f
DllMain	0.28s	0.29s	0.61%	0.62%	atio6axx	[unknown]	0	0x7ffdf9520b96
draw_chunk	0.00s	0.28s	0.00%	0.61%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	372	0x7ff7ced8713d
draw_triangles_3d_ao	0.01s	0.28s	0.02%	0.61%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	298	0x7ff7ced86b2d
delete_chunks	0.01s	0.10s	0.02%	0.22%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1236	0x7ff7ced8b25a
chunked	0.03s	0.08s	0.06%	0.18%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	160	0x7ff7ced86059
chunk_distance	0.05s	0.05s	0.11%	0.11%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	539	0x7ff7ced87c1d
floorf	0.00s	0.04s	0.01%	0.08%	craft	[unknown]	0	0x7ff7cee714aa
render_signs	0.00s	0.03s	0.01%	0.07%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1656	0x7ff7ced8cd74
floor	0.03s	0.03s	0.07%	0.07%	craft	[unknown]	0	0x7ff7cee75330
roundf	0.02s	0.03s	0.04%	0.07%	craft	[unknown]	0	0x7ff7cee6c7c1
render_wireframe	0.00s	0.03s	0.00%	0.06%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1741	0x7ff7ced8d5f9
hit_test	0.00s	0.03s	0.00%	0.06%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	647	0x7ff7ced88331
set_matrix_3d	0.00s	0.01s	0.00%	0.03%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	222	0x7ff7ced940d7
ceilf	0.01s	0.01s	0.02%	0.02%	craft	[unknown]	0	0x7ff7cee713ae
_hit_test	0.00s	0.01s	0.00%	0.02%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	614	0x7ff7ced8812d

Figure 1: Initial profiling recording.

2 Optimizations

We have implemented and tested multiple optimizations in order to improve the performance of the program. This section covers the successful optimizations, resulting in more frames per second. Before and after each optimization, we ran very sleepy to test the increase in performance and detect new bottlenecks.

2.1 Hashmap lookup

The first part of the hot path we have looked at was the find_chunk() function. The application has an array containing all current chunks and a counter for the total active chunks at a given moment. The find chunk function would loop through this array and compare the chunks x and z coordinates until it found a match. We improved this function by using a hash map instead, keeping track of the location of the chunks. As a key, we use the x and z coordinate. The hashmap we used is "uthash" [2]. While this definitely helped increase the performance, it still remains to be one of the processes with the most CPU time. We tried to further decrease this by implementing our own hash map and by increasing the amount of buckets, but neither seemed to improve performance.

Overall, this change seemed to improve performance from 6 fps to running to around 200 fps, giving us about a 30x performance increase. This also resulted in faster chunk mesh generation as a result, as the generation has to check nearby chunks for face generation.

When we look at the profiling results after implementing the hashmap in Figure 2, we can see that it now only uses 5% of the processing power. The largest bottlenecks now are processes from atio6axx, which is an executable from OpenGL. Showing us that we should optimize the OpenGL calls to the GPU if possible. The largest CPU side function is chunk_visible, which takes up about 8% of the processing time.

BaseThreadInitThunk	0.00s	187.81s	0.00%	99.99%	KERNEL32	[unknown]	0	0x7ffe8bdb257d
DllMain	35.38s	152.99s	18.84%	81.45%	atio6axx	[unknown]	0	0x7ffdf948c27d
eglGetProcAddress	0.42s	125.44s	0.22%	66.78%	atio6axx	[unknown]	0	0x7ffdfb10fdd0
RtlUserThreadStart	0.00s	125.22s	0.00%	66.67%	ntdll	[unknown]	0	0x7ffe8c9caf28
SwitchToThread	95.56s	95.56s	50.87%	50.87%	KERNEL...	[unknown]	0	0x7ffe8a12bf40
main	0.00s	62.61s	0.00%	33.33%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	2859	0x7ff6d6b330b8
mainCRTStartup	0.00s	62.59s	0.00%	33.32%	craft	[unknown]	0	0x7ff6d6b2113d
__tmainCRTStartup	0.00s	62.59s	0.00%	33.32%	craft	[unknown]	0	0x7ff6d6b2142a
render_chunks	0.27s	36.46s	0.15%	19.41%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1675	0x7ff6d6b2dd09
DrvSetCallbackProcs	1.85s	20.95s	0.99%	11.15%	atio6axx	[unknown]	0	0x7ffdf9d3f680
glfwSwapBuffers	0.00s	18.46s	0.00%	9.83%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	548	0x7ff6d6c01f52
_glfwPlatformSwapBuffers	0.00s	18.46s	0.00%	9.83%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	603	0x7ff6d6c0be3f
SwapBuffers	0.00s	18.39s	0.00%	9.79%	gdi32full	[unknown]	0	0x7ffe89fce58e
wglSwapBuffers	0.00s	18.35s	0.00%	9.77%	OPENGL...	[unknown]	0	0x7ffdfddd1aab
_DrvSwapBuffers	0.00s	18.28s	0.00%	9.73%	OPENGL...	[unknown]	0	0x7ffdfddb57bf
DrvRealizeLayerPalette	0.00s	18.22s	0.00%	9.70%	atio6axx	[unknown]	0	0x7ffdf9d2efe5
WaitForMultipleObjects	18.12s	18.12s	9.64%	9.64%	KERNEL...	[unknown]	0	0x7ffe8a115f8e
chunk_visible	15.01s	15.01s	7.99%	7.99%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	600	0x7ff6d6b282a4
ensure_chunks	0.00s	11.58s	0.00%	6.17%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1483	0x7ff6d6b2d24d
ensure_chunks_worker	1.32s	11.57s	0.71%	6.16%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1400	0x7ff6d6b2cc59
find_chunk	9.29s	10.20s	4.95%	5.43%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	577	0x7ff6d6b27d3d
draw_chunk	0.03s	9.32s	0.02%	4.96%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	417	0x7ff6d6b272d9
draw_chunk_triangles_3d_ao	0.17s	9.29s	0.09%	4.95%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	310	0x7ff6d6b26b5b
delete_chunks	0.38s	4.78s	0.20%	2.55%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1292	0x7ff6d6b2c1a8
chunked	1.16s	3.78s	0.62%	2.01%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	165	0x7ff6d6b26092
chunk_distance	2.04s	2.04s	1.09%	1.09%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	594	0x7ff6d6b2825c
floorf	0.48s	1.97s	0.25%	1.05%	craft	[unknown]	0	0x7ff6d6c12a07

Figure 2: Profiling after changing to a hashmap for the find$_chunkfunction$

2.2 SIMD frustum culling

While profiling using very sleepy, we found that another expensive function was the frustum culling. Before rendering a chunk, this function checks whether the bounding box is inside or outside the frustum of the player, decreasing the amount of draw calls by culling chunks outside the view of the player. This is done for every active chunk, which are a lot of chunks when we have a render distance of 64.

```c
int chunk_visible(float planes[6][4], int p, int q, int miny, int maxy) {
    int x = p * CHUNK_SIZE - 1;
    int z = q * CHUNK_SIZE - 1;
    int d = CHUNK_SIZE + 1;
    float points[8][3] = {
        {x + 0, miny, z + 0},
        {x + d, miny, z + 0},
        {x + 0, miny, z + d},
        {x + d, miny, z + d},
        {x + 0, maxy, z + 0},
        {x + d, maxy, z + 0},
        {x + 0, maxy, z + d},
        {x + d, maxy, z + d}
    };
    int n = g->ortho ? 4 : 6;
    for (int i = 0; i < n; i++) {
        int in = 0;
        int out = 0;
        for (int j = 0; j < 8; j++) {
            float d =
                planes[i][0] * points[j][0] +
                planes[i][1] * points[j][1] +
                planes[i][2] * points[j][2] +
                planes[i][3];
            if (d < 0) {
                out++;
            }
            else {
                in++;
                break;
            }
        }
        if (in == 0) {
            return 0;
        }
    }
    return 1;
}
```

(a) Original

```c
int chunk_visible(__m256 planes[6][4], int p, int q, int miny, int maxy) {
    int x = p * CHUNK_SIZE - 1;
    int z = q * CHUNK_SIZE - 1;
    int d = CHUNK_SIZE + 1;
    const __m256 d1 = _mm256_set_ps(0,d,0,d,0,d,0,d);
    const __m256 d2 = _mm256_set_ps(0,0,d,d,0,0,d,d);

    __m256 points_x = _mm256_add_ps(_mm256_set1_ps(x), d1);
    __m256 points_y = _mm256_set_ps(miny,miny,miny,miny,maxy,maxy,maxy,maxy);
    __m256 points_z = _mm256_add_ps(_mm256_set1_ps(z), d2);

    int n = g->ortho ? 4 : 6;
    for (int i = 0; i < n; i++) {
        __m256 res_x = _mm256_mul_ps(planes[i][0], points_x);
        __m256 res_y = _mm256_mul_ps(planes[i][1], points_y);
        __m256 res_z = _mm256_mul_ps(planes[i][2], points_z);

        __m256 res_tot = _mm256_add_ps(_mm256_add_ps(_mm256_add_ps(res_x, res_y), res_z), planes[i][3]);
        __m256 res_mask = _mm256_cmp_ps(res_tot, _mm256_setzero_ps(), _CMP_GE_OQ);
        int mask = _mm256_movemask_ps(res_mask);
        if(mask == 0)
            return 0;
    }
    return 1;
}
```

(b) SIMD

Figure 3: Frustum culling methods

After inspecting the frustum culling function, we noticed it was doing a convenient for-loop of 8 iterations over each corner point of the chunk bounding box. We thought that this was a perfect opportunity to apply SIMD with AVX1.

In contrast to our expectations, there was almost no difference in performance between using the original and new frustum culling function. This was because the majority of performance costs came from the GPU. To test whether the frustum culling function was indeed slower or faster, we ran a quick test by turning off rendering and simply running the original and SIMD code 1.000.000x times, and then printing the results so nothing gets optimized out. Without SIMD it ran at roughly 142 FPS, and with SIMD at 281 FPS. While the increase in performance was quite significant, the frustum culling method was simply not really a bottleneck anymore, in comparison to the massive speedup we got from applying the hashmap. Thus from now on, we decided to focus fully on decreasing the work the GPU needed to do.

2.3 Vertex Data compression

After drastically improving the performance of everything on the CPU side by improving the chunk lookup function, we were left with most of our bottleneck being the OpenGL rendering pipeline.

When we looked at the vertex data being sent to the GPU, we could see that 10 floats were being used for every vertex. 3 floats for the position, 3 for the normal, 2 for the UV, 1 float for light and the final float for ambient occlusion.

We managed to pack the position into 24 bits by storing the relative position to the start of the chunk in integer value. As the blocks are a set size, we do not need floating point precision. They will always be aligned to the grid. The normal vector also had 10 possible values, which allowed us to use an enum. 6 for the different faces, and the final 4 for the plant faces, which are crossed diagonally.

This allowed us to pack the position and normal data inside of one unsigned integer, instead of 6 separate floats.

As all the blocks were stored in one texture, we could also use the same method to use less bits for the UV value. The texture map has room for 16 tiles in both the x and y direction. We could simply calculate the float value by multiplying the index of the texture by (1/16). Note that the left vertices would have the index, while the right vertices would have that of the next index, to create the proper uv mapping.

The light value ranged from 0 to 1 in steps of (1/15) allowing us to use integer values that would get divided by the vertex shader. The ambient occlusion variable was either 0, 0.5, or 1. Allowing for easy mapping as well.

We ended up using 1 byte for every individual value, giving us a structure of 64 bits per vertex. While it was possible to pack the values more closely, it would not result in better performance. Unfortunately containing everything inside of 32 bits was just barely impossible. Especially with future optimizations we were thinking off.

In order to get the global position again from the chunk position, we are passing in the chunk location as a uniform for every chunks draw call.

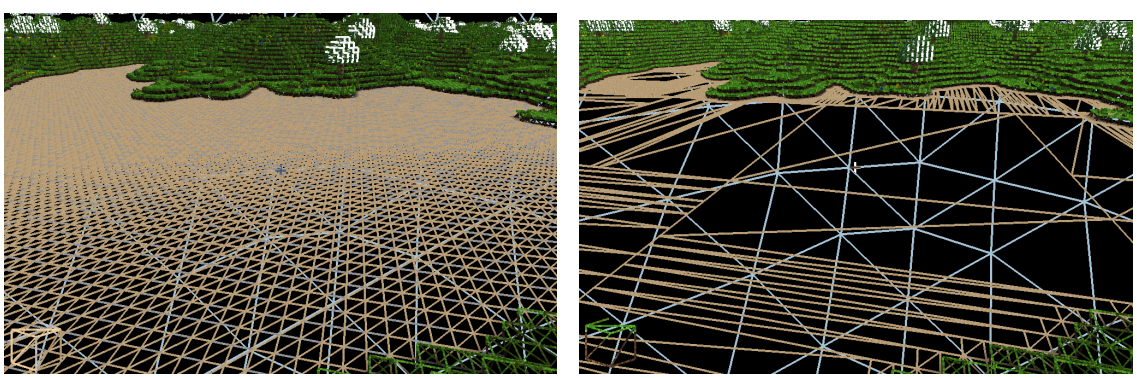
This optimization resulted in about double the fps, going from 200 to around 400 fps. When we look at the new profiling results in Figure 4, we can see that the BufferSwap calls do not show up anymore and that the percentages on the atio6axx functions have decreased, but they are still the main bottleneck.

BaseThreadInitThunk	0.00s	155.01s	0.00%	99.98%	KERNEL32	[unknown]	0	0x7ffe8bdb257d
DllMain	40.35s	112.64s	26.02%	72.65%	atio6axx	[unknown]	0	0x7ffdf948c27d
eglGetProcAddress	1.43s	103.45s	0.92%	66.72%	atio6axx	[unknown]	0	0x7ffdfb10fdd0
RtlUserThreadStart	0.00s	103.36s	0.00%	66.67%	ntdll	[unknown]	0	0x7ffe8c9caf28
SwitchToThread	65.51s	65.51s	42.26%	42.26%	KERNEL...	[unknown]	0	0x7ffe8a12bf40
main	0.00s	51.67s	0.00%	33.32%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	2936	0x7ff7b4d13c9f
mainCRTStartup	0.00s	51.65s	0.00%	33.32%	craft	[unknown]	0	0x7ff7b4d0113d
__tmainCRTStartup	0.00s	51.65s	0.00%	33.32%	craft	[unknown]	0	0x7ff7b4d0142a
render_chunks	0.50s	40.99s	0.32%	26.44%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1751	0x7ff7b4d0e812
ensure_chunks	0.00s	14.78s	0.00%	9.53%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1557	0x7ff7b4d0dd70
ensure_chunks_worker	1.72s	14.77s	1.11%	9.52%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1475	0x7ff7b4d0d78a
find_chunk	11.58s	12.97s	7.47%	8.37%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	586	0x7ff7b4d07af7
draw_chunk	0.09s	9.24s	0.06%	5.96%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	424	0x7ff7b4d0703c
chunk_visible	9.17s	9.17s	5.92%	5.92%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	604	0x7ff7b4d0805d
draw_chunk_triangles_3d_ao	0.19s	8.88s	0.12%	5.73%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	309	0x7ff7b4d06926
delete_chunks	0.62s	7.24s	0.40%	4.67%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1363	0x7ff7b4d0c146
chunk_visible	6.70s	6.70s	4.32%	4.32%	craft	c:/programdata/chocolatey/lib/winlibs/tools/...	1252	0x7ff7b4d0814f
chunked	1.33s	5.68s	0.86%	3.66%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	169	0x7ff7b4d05ea2
DrvSetCallbackProcs	3.34s	4.73s	2.16%	3.05%	atio6axx	[unknown]	0	0x7ffdf9d3f6af
floorf	0.88s	3.20s	0.57%	2.06%	craft	[unknown]	0	0x7ff7b4df3544
chunk_distance	2.87s	2.87s	1.85%	1.85%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	597	0x7ff7b4d08015
floor	2.32s	2.32s	1.50%	1.50%	craft	[unknown]	0	0x7ff7b4df73cb
boost::archive::detail::iserializer<sc_xml_iarchive,_SC_SI_PS_AT...	0.50s	2.10s	0.32%	1.35%	atio6axx	[unknown]	0	0x7ffdfab5674a
roundf	1.13s	1.68s	0.73%	1.08%	craft	[unknown]	0	0x7ff7b4dee860
render_wireframe	0.00s	1.54s	0.00%	0.99%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1879	0x7ff7b4d0f510
hit_test	0.17s	1.53s	0.11%	0.98%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	765	0x7ff7b4d089c2
memcmp	1.39s	1.39s	0.90%	0.90%	msvcrt	[unknown]	0	0x7ffe8b1f1b60

Figure 4: Profiling result after only using 64 bits per vertex

2.4 Greedy meshing

Next to compressing the vertex data, another way to decrease the amount of vertex data is by simply needing less vertex data!



(a) Original Mesher (b) Greedy Mesher

Figure 5: Before and after of applying a greedy mesher

Out of the box, the application was using a standard culled voxel-mesher, where each voxel can consist of a max of 6 faces for each direction. While the mesh generation does cull faces inside

the voxel volume, There are still large flat surfaces that could easily be compressed into a single face, as seen in figure 5a.

A technique called greedy meshing can be used to optimize this. The basic idea of the algorithm is that you check for each voxel in a set direction whether the next voxel is of the same block type and orientation. Once it reaches a culled voxel or another voxel block type (or in this case, also a block with different lighting or ambient occlusion), we expand the row of similar voxels in a perpendicular direction, checking if each voxel is of the same type as well. Once the row is different or we reach the edge of a chunk, we generate a single larger face out of the combined voxels.

The only draw-back to this technique is that chunk generation is slightly slower now. This is acceptable though, since we are focusing on optimizing the rendering scheme. Of course when placing blocks, this does result in slightly larger lag spikes. This can easily be solved by simply using the original mesher when editing terrain. We can then asynchronously do the greedy mesher on a separate thread, eventually resulting in the optimized mesh without lag spikes.

The result of the greedy mesher can be seen in figure 5b. Since the world generation results in a lot of flat sand areas, a great amount of faces are combined into singular faces, decreasing the total amount of vertex data even more.

The optimization results can be seen in the last subsection, since they varied quite a bit.

2.5 Using Indices

One obvious issue with the original meshes were that they consisted of only vertex data. When drawing the mesh, a single vertex buffer is send to be drawn by the GPU, where every 3 vertices next to each other in the buffer represents a single triangle. The issue with this is that all faces consist of 6 vertices, 2 of which are duplicates.

For this reason, renderers normally use indices buffers that define which vertices make up which triangles. For a single face, you would need just 4 vertices, and 6 indices which point to the location of the vertices in the vertex buffer. This greatly reduces the amount of vertex data even more.

2.6 Comparison

We got some interesting results comparing the before and after of applying greedy meshing and indices. We have very different GPU's, and got varying results from these optimisations.

	Before	After
AMD Radeon RX 6800	425	433
AMD Radeon 5700XT	102	120

Table 1: FPS before and after implementing greedy meshing and indices.

The total amount of vertices went down from 22.432.710 to 17.846.320. From the results above, you can see how the higher-end GPU has very little performance gain, while the lower end GPU relatively gains a lot more from the decrease in vertices. This could be explained by the fact that the bottleneck for the higher-end PC comes from the amount of draw calls happening to render all visible chunks.

When we look at the final profiling results in figure 6, we see no big differences between the previous and this result. The CPU functions do seem to have a slightly higher percentage however, indicating that the GPU load has somewhat decreased. The profiling is done with the higher-end GPU, the percentages might have changed more drastically on the lower-end GPU.

Name	Excl...	Incl...	% Exclusi...	% Inclusi...	Module	Source File	So...	Address
BaseThreadInitThunk	0.00s	90.97s	0.00%	99.99%	KERNEL32	[unknown]	0	0x7ffe8bdb257d
DllMain	33.66s	68.18s	37.00%	74.94%	atio6axx	[unknown]	0	0x7ffdf948c7b3
eglGetProcAddress	1.98s	60.71s	2.18%	66.73%	atio6axx	[unknown]	0	0x7ffdfb10fdb1
RtlUserThreadStart	0.00s	60.65s	0.00%	66.67%	ntdll	[unknown]	0	0x7ffe8c9caf28
main	0.01s	30.33s	0.01%	33.33%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	3424	0x7ff6cb4c68ee
mainCRTStartup	0.00s	30.32s	0.00%	33.32%	craft	[unknown]	0	0x7ff6cb4b113d
__tmainCRTStartup	0.00s	30.32s	0.00%	33.32%	craft	[unknown]	0	0x7ff6cb4b142a
SwitchToThread	29.67s	29.67s	32.61%	32.61%	KERNEL...	[unknown]	0	0x7ffe8a12bf40
render_chunks	0.31s	24.62s	0.34%	27.06%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	2170	0x7ff6cb4c1447
ensure_chunks	0.00s	8.19s	0.00%	9.00%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1976	0x7ff6cb4c0a03
ensure_chunks_worker	0.89s	8.17s	0.97%	8.98%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1894	0x7ff6cb4c041d
draw_chunk	0.04s	7.55s	0.04%	8.30%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	462	0x7ff6cb4b8486
draw_chunk_triangles_3d_ao_new	0.16s	7.37s	0.17%	8.11%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	345	0x7ff6cb4b7c0f
find_chunk	6.59s	7.25s	7.24%	7.97%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	624	0x7ff6cb4b8f47
chunk_visible	4.81s	4.81s	5.28%	5.28%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	642	0x7ff6cb4b917e
delete_chunks	0.31s	3.60s	0.34%	3.96%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	1782	0x7ff6cb4bf12b
chunk_visible	3.48s	3.48s	3.83%	3.83%	craft	c:/programdata/chocolatey/lib/winlibs/tools/...	1252	0x7ff6cb4b9270
chunked	0.66s	2.77s	0.72%	3.04%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	205	0x7ff6cb4b7192
DrvSetCallbackProcs	1.98s	2.46s	2.17%	2.70%	atio6axx	[unknown]	0	0x7ffdf9d3f6af
chunk_distance	1.58s	1.58s	1.74%	1.74%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	635	0x7ff6cb4b9136
floorf	0.43s	1.58s	0.47%	1.74%	craft	[unknown]	0	0x7ff6cb5b99f0
floor	1.15s	1.15s	1.27%	1.27%	craft	[unknown]	0	0x7ff6cb5bd871
render_wireframe	0.00s	0.84s	0.00%	0.93%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	2353	0x7ff6cb4c21c8
hit_test	0.12s	0.84s	0.13%	0.92%	craft	C:/Users/roeld/Documents/git/Uni/UU/Optimi...	764	0x7ff6cb4b9a7e
boost::archive::detail::iserializer<sc_xml_iarchive,_SC_SI_PS_AT...	0.29s	0.82s	0.32%	0.90%	atio6axx	[unknown]	0	0x7ffdfab56745
roundf	0.50s	0.81s	0.55%	0.89%	craft	[unknown]	0	0x7ff6cb5b4cb0
glDrawElements	0.02s	0.78s	0.02%	0.85%	OPENGL...	[unknown]	0	0x7ffdfddb75ce

Figure 6: Profiling result after implementing greedy meshing and indices.

3 Failed attempts

3.1 Frustum Culling on the GPU

We came up with an interesting way to utilize a compute shader to optimize the frustum culling, which we simply wanted to try implementing. The idea behind it goes as followed: We keep track of a grid of integers, the width and size of the render distance, around the player. For every tile on the grid, we can check if the tile is inside or outside the frustum in parallel using a compute shader. We then read the grid back to the CPU, and get the relevant result of each chunk by mapping the chunks position relative to the player to the generated frustum grid.

When we profiled the resulting code, we saw that the read back from the GPU was now taking up most of the performance, which decreased our overall fps. We tried several things to mitigate this.

We first tried decreasing the amount of data by tracking a grid of chars instead of integers. This meant we had to check 4 grid elements per GPU thread, since char buffers aren't supported. Since we had to pack the results into integers with bit shifting, we would also have to write to the same addresses, which isn't a good idea for a compute buffer.

With less data needing to be read back, we didn't see much improvement in the performance. We thus decided to look into reading the data back asynchronously. This didn't save much performance either, so we had to leave it as is due to time constraints.

We do think it could improve performance if we could effectively asynchronously read back the data (maybe reading data back from a separate thread). Some issues do arise with this implementation. When quickly rotating the camera, you can see some of the chunks being culled in the corner of the screen, due to the fact that the up-to-date frustum culling data isn't read back yet in the relevant frame. This could be solved by increasing the FOV of the player, adding a slight padding for scenarios like this.

3.2 Profiling the GPU

As the biggest part of our bottleneck now was rendering the triangles, we tried to look into profiling the GPU. Unfortunately we weren't able to use the AMD GPU profiler as it does not support OpenGL. We did manage to use RenderDoc after updating the GLFW library, but we were not able to gain any meaningful insights. We concluded that the amount of draw calls was

a big bottleneck from testing bigger chunks with a lower chunk radius, which resulted in better performance, supporting our suspicion.

4 Conclusion

Our final optimizations brought us a Minecraft clone where we can render a render distance of 64 chunks at a stable FPS. We applied both high-level optimizations, like applying hash-maps or greedy meshing, and low-level optimizations, like compressing vertex data. While the result is about 50 times faster, there is still a lot that could be optimized.

Optimizing the amount of draw calls would be the next thing to focus on. This could be done by combining chunks into larger meshes, which would increase the amount of vertices per chunk but decreasing the draw calls. Another high-level optimization would be to have some sort of level-of-detail system, which renders larger less-detailed chunks in the distance, by using an octree for example. This can look odd though, since lowering the detail of a chunk would mean the blocks would be x8 larger. We have also read about ways to render the chunks in a single draw call, but we're not sure how.

Focusing on speeding up the chunk generation could also be interesting. Applying GPGPU on the mesh generation is very much possible, which can be done by generating each voxel in parallel. This could reduce the lag spikes that happen when you place or break blocks.

Figure 7: Thanks!

Above you can see the final result of the app, working as intended. Thanks for the lectures and this great course!

References

[1] Craft, https://github.com/fogleman/Craft.

[2] uthash, https://troydhanson.github.io/uthash/.